

DẠNG BÀI TẬP: merkel tree: xây dựng tree, duyệt, thêm, bảng ascii coi cột hệ 10 và đồ họa

Quy ước của thầy bạn

Khi “nói” h1 || h2 ra chuỗi "6566"

Nhưng khi hash, không lấy ASCII của ký tự '6','5'...

Mà lấy trực tiếp từng chữ số và cộng chữ số lại (digit-sum), rồi (nếu đề yêu cầu) mới mod.

mod 120? nghĩa là gì? mod 120 = lấy phần dư khi chia cho 120. Tức là: 215 chia cho

120 dư bao nhiêu?

32= khoảng trống, 33=!, 34=", 35=#, 36=\$, 37=%, 38=&, 39=', 40=(, 41=), 42=*, 43=+, 44=., 45=-, 46=., 47=/, 48=0, 49=1, 50=2, 51=3, 52=4, 53=5, 54=6, 55=7, 56=8, 57=9, 58=:, 59=;, 60=<, 61==, 62=>, 63=?, 64=@, 65=A, 66=B, 67=C, 68=D, 69=E, 70=F, 71=G, 72=H, 73=I, 74=J, 75=K, 76=L, 77=M, 78=N, 79=O, 80=P, 81=Q, 82=R, 83=S, 84=T, 85=U, 86=V, 87=W, 88=X, 89=Y, 90=Z, 91=[, 92=\, 93=], 94=^, 95=_, 96=`, 97=a, 98=b, 99=c, 100=d, 101=e, 102=f, 103=g, 104=h, 105=i, 106=j, 107=k, 108=l, 109=m, 110=n, 111=o, 112=p, 113=q, 114=r, 115=s, 116=t, 117=u, 118=v, 119=w, 120=x, 121=y, 122=z, 123={, 124=|, 125=}, 126=~

Bài 1: Cho ví dụ một block có 4 giao dịch (Tx1, Tx2, Tx3, Tx4). Hãy xây dựng cây Merkle Root.

Biết giá trị của các giao dịch như sau: Tx1 = "A"; Tx2 = "B"; Tx3 = "C"; Tx4 = "D".

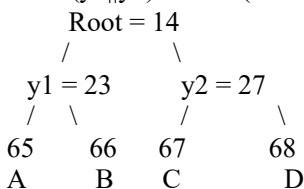
- Xây dựng Merkle tree cho 4 giao dịch.
- Tính hash lá: $h_i = \text{Hash}(Tx_i)$, với Hash là tổng mã ASCII mod 120.
- Tính mức giữa: $y_1 = \text{Hash}(h_1||h_2)$, $y_2 = \text{Hash}(h_3||h_4)$. (nói giá trị rồi tính tiếp).
- Tính Merkle root: $\text{Root} = \text{Hash}(y_1||y_2)$.
- Vẽ cây.
- Nếu thêm Tx5 = "BC", thì merkel tree sẽ được vẽ như thế nào?
- Thực hiện kiểm tra Tx6 = "E" có tồn tại trong merkel tree?

1) Xây dựng Merkle tree cho 4 giao dịch (Tx1..Tx4) Cho: Tx1="A", Tx2="B", Tx3="C", Tx4="D"

Hash lá: $h_1 = \text{Hash}("A") = 65$. $h_2 = \text{Hash}("B") = 66$. $h_3 = \text{Hash}("C") = 67$. $h_4 = \text{Hash}("D") = 68$

Mức giữa: $y_1 = \text{Hash}(h_1||h_2) = \text{Hash}("6566") = 6+5+6+6 = 23$. $y_2 = \text{Hash}(h_3||h_4) = \text{Hash}("6768") = 6+7+6+8 = 27$

Root = $\text{Hash}(y_1||y_2) = \text{Hash}("2327") = 2+3+2+7 = 14$



2) Nếu thêm Tx5 = "BC" thì Merkle tree vẽ thế nào.

Bước 1 — Hash lá cho Tx5. Tx5 = "BC". ASCII(B)=66, ASCII(C)=67 $\Rightarrow$ tổng = 133. $h_5 = 133 \bmod 120 = 13$. Bây giờ có 5 lá (lẻ) $\Rightarrow$ lặp lá cuối để đủ chẵn: $h_6 = h_5 = 13$

Bước 2 — Tầng giữa (ghép cặp, nối rồi cộng chữ số)

$y_1 = \text{Hash}("65||66") = \text{Hash}("6566") = 23$

$y_2 = \text{Hash}("67||68") = \text{Hash}("6768") = 27$

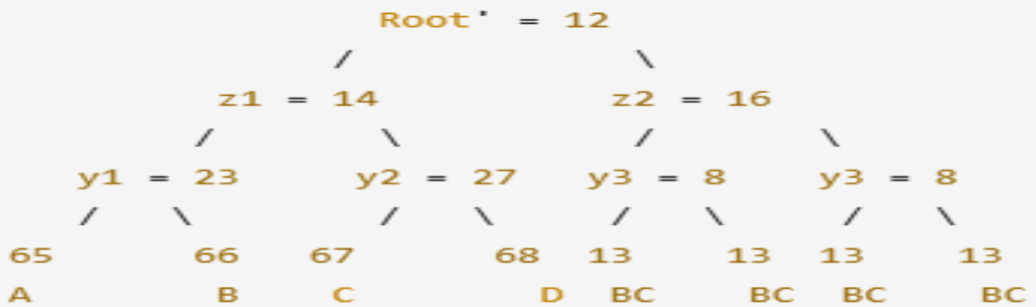
$y_3 = \text{Hash}("13||13") = \text{Hash}("1313") = 1+3+1+3 = 8$

Bước 3 — Tầng trên. Vì tầng này có 3 nút (lẻ), lại lặp nút cuối:

$z_1 = \text{Hash}(y_1||y_2) = \text{Hash}("23||27") = \text{Hash}("2327") = 14$

$z_2 = \text{Hash}(y_3||y_3) = \text{Hash}("8||8") = \text{Hash}("88") = 8+8 = 16$

Bước 4 — Root mới. Root' = $\text{Hash}(z_1||z_2) = \text{Hash}("14||16") = \text{Hash}("1416") = 1+4+1+6 = 12$. Kết quả khi thêm Tx5="BC": Root' = 12



3) Kiểm tra Tx6 = "E" có tồn tại trong Merkle tree không? Tx6 = "E" $\Rightarrow$ ASCII("E") = 69. $h6? = 69 \bmod 120 = 69$

So với các lá ban đầu: Với 4 tx: {65, 66, 67, 68} $\rightarrow$ không có 69. Với 5 tx (có BC): {65, 66, 67, 68, 13} $\rightarrow$ không có 69

Kết luận: Tx6 = "E" không tồn tại trong Merkle tree (không thuộc danh sách giao dịch).

Bài 2: (phiên bản lá lẻ ngay từ đầu) Cho 1 block có 3 giao dịch: Tx1 = "Huynh Ba Khang" Tx2 = "B320" Tx3 = "Cong nghe Blockchain" Biết: Hash(Tx) = (tổng ASCII của từng ký tự trong Tx) mod 120. Nếu số lá lẻ $\Rightarrow$ lặp lá cuối. Hash nút trong: nối hai giá trị rồi cộng chữ số (như 6566 $\rightarrow$ 23)

Yêu cầu: Xây dựng Merkle tree và tính Merkle Root cho 3 giao dịch. Nếu thêm Tx4 = "BK" thì Root mới là bao nhiêu? Kiểm tra Tx5 = "E" có tồn tại trong Merkle tree không?

(1) Merkle Root khi ban đầu có 3 lá (lẻ)

Bước 1 — Hash lá (ASCII mod 120) (tính luôn khoảng trống - 32 -

$h1 = \text{Hash}(\text{"Huynh Ba Khang"}) = 1240 \bmod 120 = 40$

$h2 = \text{Hash}(\text{"B320"}) = 215 \bmod 120 = 95$

$h3 = \text{Hash}(\text{"Cong nghe Blockchain"}) = 1879 \bmod 120 = 79$

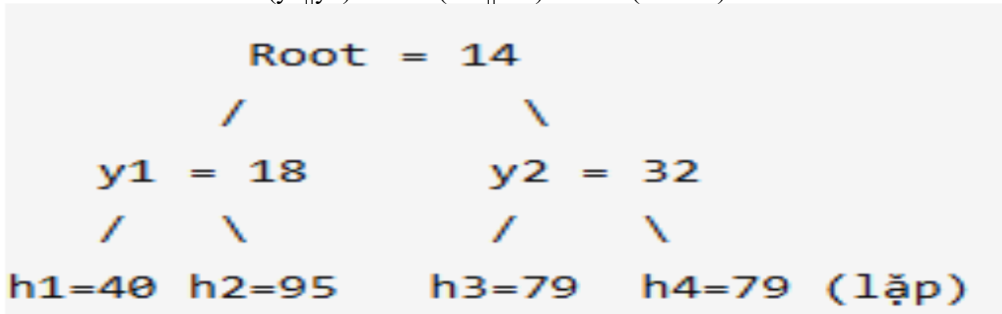
Vì 3 lá lẻ $\Rightarrow$ lặp lá cuối: $h4 = h3 = 79$

Bước 2 — Tầng giữa (nối rồi cộng chữ số)

$y1 = \text{Hash}(h1||h2) = \text{Hash}(\text{"40||95"}) = \text{Hash}(\text{"4095"}) = 4+0+9+5 = 18$

$y2 = \text{Hash}(h3||h4) = \text{Hash}(\text{"79||79"}) = \text{Hash}(\text{"7979"}) = 7+9+7+9 = 32$

Bước 3 — Root = Hash(y1|y2) = Hash("18|32") = Hash("1832") = 1+8+3+2 = 14



(2) Thêm Tx4 = "BK" $\rightarrow$ Root mới

Hash lá mới: $h4 = \text{Hash}(\text{"BK"}) = (66+75)=141 \bmod 120 = 21$ (lúc này có 4 lá nên không cần lặp)

Tầng giữa: $y1 = \text{Hash}(\text{"40||95"}) = \text{Hash}(\text{"4095"}) = 18$. $y2 = \text{Hash}(\text{"79||21"}) = \text{Hash}(\text{"7921"}) = 7+9+2+1 = 19$

Root mới: Root' = Hash("18|19") = Hash("1819") = 1+8+1+9 = 19

(3) Tx5 = "E" có tồn tại không? Hash("E") = 69 mod 120 = 69. Tập hash lá đang có (sau khi thêm BK): {40, 95, 79, 21} $\Rightarrow$ không có 69 Tx5 = "E" không tồn tại trong Merkle tree (theo cách kiểm tra giống mẫu trong file)

confirmations => Rủi ro bị đánh lừa là rất cao. **5. Kết luận – Trả lời trực tiếp câu hỏi** Vì sao chiến lược này mang rủi ro lớn? Vì: Một giao dịch nằm trong 1 block vẫn có thể bị đảo ngược.Bitcoin không đảm bảo finality ngay lập tức.Website gửi hàng trước khi giao dịch thực sự bất biến=>Nguy cơ mất hàng mà không nhận được tiền.

BÀI 1: Trình bày mục tiêu của SPV (Simplified Payment Verification) - trong slide B2. **Lời giải:** Mục tiêu của SPV là cho phép một client nhẹ xác minh rằng một giao dịch đã được ghi vào blockchain mà không cần tải và lưu toàn bộ blockchain. SPV đạt được điều này bằng cách chỉ lưu block header và sử dụng Merkle proof để kiểm tra sự tồn tại của giao dịch trong một block hợp lệ, từ đó giảm đáng kể chi phí lưu trữ và tính toán.

BÀI 2: So sánh node đầy đủ (Full Node) và SPV client - TRONG SLIDE B2. **Lời giải:**

Đặc điểm	Full Node (Node đầy đủ)	SPV Client (Ví nhẹ)
Lưu trữ	Lưu trữ toàn bộ dữ liệu blockchain (ví dụ: ~366GB).	Chỉ lưu trữ Block Headers (dung lượng rất nhỏ).
Bảo mật	Tự xác thực mọi quy tắc của mạng lưới, không cần tin tưởng ai khác.	Phải dựa vào (tin tưởng) một phần vào các server/node đầy đủ khác.
Quyền riêng tư	Tốt nhất, vì node sở hữu toàn bộ dữ liệu và không phải hỏi node khác về địa chỉ của mình.	Kém hơn, vì phải gửi thông tin địa chỉ/Bloom filter cho server để yêu cầu dữ liệu.
Tốc độ/Hiệu năng	Đòi hỏi phần cứng mạnh, thời gian đồng bộ rất lâu.	Hoạt động nhanh, phù hợp cho thiết bị di động.

BÀI 3: Nếu một SPV client kết nối đến server độc hại, có thể gặp rủi ro gì? - TRONG SLIDE B2. **Lời giải:**Nếu SPV client kết nối đến server độc hại, client có thể bị cung cấp thông tin sai lệch, chẳng hạn như che giấu giao dịch hợp lệ, trì hoãn xác nhận hoặc dẫn dắt client tin rằng một giao dịch chưa hoặc đã được xác nhận sai sự thật. Ngoài ra, server độc hại có thể theo dõi các truy vấn để suy ra lịch sử giao dịch, gây rủi ro về quyền riêng tư của người dùng.

BÀI 4: Mô tả cách Electrum cải thiện SPV để tăng bảo mật và quyền riêng tư. - TRONG SLIDE B2. **Lời giải:** Electrum cải thiện SPV bằng cách cho phép client kết nối đến nhiều server thay vì chỉ một server duy nhất, từ đó giảm nguy cơ bị kiểm soát hoặc cung cấp thông tin sai. Ngoài ra, Electrum sử dụng cơ chế truy vấn theo script hash thay vì địa chỉ trực tiếp, giúp hạn chế việc server suy ra toàn bộ lịch sử giao dịch của người dùng, qua đó nâng cao quyền riêng tư.

BÀI 5: (Case study) Giả sử bạn phát triển một ví Bitcoin nhẹ chạy trên điện thoại. Hãy đề xuất cách áp dụng SPV để giảm tải, đồng thời phân tích trade-off giữa bảo mật và hiệu năng. - TRONG SLIDE B2. **Lời giải:** Khi phát triển một ví Bitcoin nhẹ trên điện thoại, SPV có thể được áp dụng để giảm tải bằng cách chỉ lưu block header và sử dụng Merkle proof để xác minh giao dịch, giúp tiết kiệm bộ nhớ, pin và băng thông. Tuy nhiên, trade-off là mức độ bảo mật thấp hơn so với full node vì ví phải tin cậy vào server bên ngoài, và quyền riêng tư có thể bị ảnh hưởng nếu server theo dõi các truy vấn. Do đó, cần cân nhắc giữa hiệu năng cao và mức độ bảo mật chấp nhận được cho người dùng di động.

Các câu TỰ bổ sung thêm:

BÀI 6: – Quy trình: Trình bày cách client nhẹ xác minh một giao dịch trong Bitcoin. (Một client nhẹ (light node) muốn xác minh rằng giao dịch Tx đã được ghi vào blockchain.Client không lưu toàn bộ blockchain.Hãy trình bày cách xác thực giao dịch và xác minh thanh toán.) **Lời giải:**

BƯỚC 1: Xác định thông tin client có: Client nhẹ KHÔNG lưu toàn bộ block, mà chỉ lưu:Block header của các block.Merkle root của mỗi block.Chuỗi hash các block (proof-of-work)

BƯỚC 2: Nhận Merkle proof của giao dịch: Để xác minh giao dịch Tx, client yêu cầu: Giao dịch Tx. Tập Merkle proof π (các hash anh em). Block header chứa Merkle root h.

BƯỚC 3: Xác minh giao dịch bằng Merkle tree: Client thực hiện: Hash giao dịch Tx. Dùng Merkle proof π để hash lại các node trung gian. Thu được Merkle root h'. So sánh: $h' = h$. Nếu đúng $\rightarrow$ giao dịch nằm trong block.

BƯỚC 4: Xác minh block hợp lệ (Proof-of-Work): Client tiếp tục kiểm tra: Hash của block header. Block đó có nằm trong chuỗi dài nhất (longest chain) hay không. Đây là điểm xác minh thanh toán: Block càng nhiều block sau nó $\rightarrow$ thanh toán càng an toàn.

BÀI 7 – SO SÁNH FULL NODE VÀ LIGHT NODE: So sánh full node và light node trong quá trình xác minh giao dịch Bitcoin. **lời giải:** Full node và light node khác nhau chủ yếu ở lượng dữ liệu lưu trữ và cách xác minh giao dịch. Full node lưu trữ toàn bộ blockchain, bao gồm tất cả các block và toàn bộ giao dịch. Khi xác minh một giao dịch, full node có thể tự kiểm tra trực tiếp: Tính hợp lệ của giao dịch. Tính đúng đắn của chữ ký số. Quy tắc UTXO (đầu vào chưa bị chi tiêu). Tính hợp lệ của block chứa giao dịch.

Do đó, full node không cần tin cậy node khác và có mức độ an toàn cao nhất. Ngược lại, light node (SPV client) không lưu toàn bộ blockchain, mà chỉ lưu block header của các block. Khi cần xác minh một giao dịch, light node không tự kiểm tra toàn bộ giao dịch trong block, mà sử dụng: Merkle proof của giao dịch. Merkle root trong block header. Chuỗi block header có proof-of-work mạnh nhất. Light node xác minh rằng giao dịch đã được đưa vào một block hợp lệ, nhưng không kiểm tra chi tiết toàn bộ nội dung block như full node. Vì vậy, light node tiết kiệm tài nguyên hơn nhưng phải tin cậy một phần vào các full node cung cấp dữ liệu. **Kết luận:** Full node xác minh giao dịch đầy đủ và độc lập, trong khi light node xác minh giao dịch gián tiếp thông qua Merkle proof và block header.

BÀI 8 – VÌ SAO SPV HIỆU QUẢ / KHÔNG CẦN TẢI TOÀN BỘ BLOCKCHAIN: Vì sao cơ chế SPV cho phép xác minh thanh toán mà không cần tải toàn bộ blockchain? **lời giải:** Cơ chế SPV (Simple Payment Verification) không cần tải toàn bộ blockchain nhờ việc sử dụng Merkle tree trong cấu trúc block của Bitcoin. Trong mỗi block, tất cả các giao dịch được tổ chức dưới dạng Merkle tree và chỉ Merkle root được lưu trong block header. Khi cần xác minh một giao dịch, client nhẹ chỉ cần: Hash của giao dịch, Merkle proof gồm các hash anh em. Block header chứa Merkle root. Nhờ cấu trúc Merkle tree, một giao dịch có thể được chứng minh là thuộc block chỉ bằng một số lượng nhỏ hash, có kích thước tỉ lệ theo $O(\log n)$, thay vì phải tải toàn bộ n giao dịch trong block. Ngoài ra, SPV client chỉ cần lưu chuỗi block header để kiểm tra proof-of-work và xác định block nằm trong chuỗi hợp lệ dài nhất. Do đó, dung lượng lưu trữ và chi phí tính toán được giảm đáng kể so với full node. **Kết luận:** SPV hiệu quả vì kết hợp Merkle proof và block header, cho phép xác minh thanh toán với chi phí thấp mà vẫn đảm bảo tính toàn vẹn dữ liệu.

BÀI 9 – ĐIỀU KIỆN ĐỂ THANH TOÁN ĐƯỢC COI LÀ AN TOÀN (BITCOIN): “Khi nào một thanh toán được xem là an toàn trong Bitcoin?”, “Ý nghĩa của việc giao dịch được nhiều block xác nhận?”, lời giải: Thanh toán được coi là an toàn khi thỏa mãn đồng thời các điều kiện sau:

Điều kiện 1: Giao dịch đã được ghi vào một block hợp lệ. Giao dịch Tx phải: Đã được đưa vào một block. Block này có Merkle root hợp lệ. Block header thỏa mãn Proof-of-Work. **Ý nghĩa:** Giao dịch không còn nằm ở mempool, mà đã được mạng chấp nhận.

Điều kiện 2: Block chứa giao dịch nằm trên chuỗi có Proof-of-Work mạnh nhất. Trong Bitcoin, chuỗi hợp lệ là: Chuỗi có tổng Proof-of-Work lớn nhất (thường gọi là *longest chain*). Giao dịch chỉ được coi là hợp lệ nếu: Block chứa giao dịch nằm trên chuỗi này. **Ý nghĩa:** Tránh trường hợp block bị orphan (bị loại khỏi chuỗi chính).

Điều kiện 3: Block chứa giao dịch có nhiều block theo sau (n confirmations). Mỗi block được thêm sau block chứa giao dịch gọi là 1 confirmation. Ví dụ: Block chứa Tx $\rightarrow$ 1 confirmation. Có 6 block theo sau $\rightarrow$ 6 confirmations

Ý nghĩa: Càng nhiều confirmation: Xác suất đảo ngược giao dịch (double-spend) càng thấp. Kẻ tấn công phải làm lại Proof-of-Work của block đó và tất cả các block sau. Thực tế: Bitcoin thường coi 6 confirmations là mức an toàn cao.

• **Hệ thống có thỏa mãn Termination không?** Có, thỏa mãn Termination (đối với các node đúng). Giải thích: Termination yêu cầu mọi node đúng phải kết thúc và đưa ra quyết định trong thời gian hữu hạn. Trong tình huống này, Node 1 và Node 2 đều nhận thông điệp cần thiết từ Node 0, cập nhật V_i và công bố quyết định $v=1$, tức là chúng đều dừng và quyết định. Node 3 bị crash nên không yêu cầu phải “termination” theo định nghĩa dành cho node đúng. Vì vậy Termination được thỏa mãn.

Bài 3: Byzantine fault & cơ chế lan truyền chữ ký (Dolev-Strong)

Mô tả: Một hệ thống có 5 node: Node 0 (Master), Node 1–4 (Slave).

- Node 0 (Byzantine) gửi giá trị $v=1$ cho Node 1, và giá trị $v=0$ cho Node 2–4.
- Node 1, sau khi nhận được thông điệp từ Master, ký $\langle 1:0,1 \rangle$ và phát cho toàn mạng.
- Node 2–4, sau khi nhận $\langle 0:0 \rangle$ từ Master, cũng ký $\langle 0:0,i \rangle$ và phát cho toàn mạng.
- Các node còn lại khi nhận thông điệp, chỉ chấp nhận nếu chuỗi chữ ký hợp lệ.
- Sau vòng lan truyền $f+1$, tất cả các node đều nhận được cả $\langle 1:0,1 \rangle$ và $\langle 0:0,i \rangle$ ($i=2,3,4$). Theo luật quyết định, hệ thống chọn giá trị đa số: $v=0$.

Câu hỏi:

• **Hệ thống có thỏa mãn Agreement không?** Có, thỏa mãn Agreement. Vì trong Dolev–Strong, các node đúng chỉ chấp nhận các thông điệp có chuỗi chữ ký hợp lệ và sau số vòng lan truyền cố định $f+1$, mọi node đúng sẽ thu được cùng một tập thông điệp hợp lệ (ở đây đều thấy cả $\langle 1:... \rangle$ và $\langle 0:... \rangle$). Khi tất cả node đúng áp dụng cùng một luật quyết định (chọn giá trị đa số), chúng sẽ đi tới cùng một quyết định cuối cùng. Do đó, các node đúng không thể quyết định khác nhau $\rightarrow$ Agreement được đảm bảo.

• **Hệ thống có thỏa mãn Validity không, nếu Node 0 là Byzantine?** Không đảm bảo (không thỏa mãn Validity trong trường hợp Master Byzantine). Validity (theo nghĩa “nếu Master tin cậy gửi v thì mọi node đúng quyết định v ”) chỉ được đảm bảo khi Master/Commander là trung thực. Ở đây Node 0 là Byzantine và cố tình gửi các giá trị khác nhau cho các node khác nhau, nên không có khái niệm “giá trị đúng duy nhất” từ Master để hệ thống phải bảo toàn. Vì vậy, giao thức ưu tiên đảm bảo Agreement giữa các node đúng; kết quả quyết định (ở đây là $v=0$) không nhất thiết phản ánh ý định hay giá trị ban đầu của Node 0 $\rightarrow$ Validity không được đảm bảo khi Master Byzantine.

• **Hệ thống có thỏa mãn Termination không?** Có, thỏa mãn Termination.

Giao thức Dolev–Strong chạy theo số vòng cố định là $f+1$. Sau khi hoàn tất số vòng này, mỗi node đúng đều có đủ thông tin (tập thông điệp hợp lệ) để áp dụng luật quyết định và công bố kết quả. Vì số vòng là hữu hạn và được định trước, các node đúng chắc chắn kết thúc và đưa ra quyết định $\rightarrow$ Termination được thỏa mãn.

Bài 4: Minh họa- trong file B3_CacGiaoThucDongThuan SLIDE 42

• **Thiết kế một kịch bản minh họa giao thức Dolev-Strong với 4 node, trong đó có 1 node Byzantine. Hãy chỉ rõ cách lan truyền chữ ký số trong từng vòng.**

Giả sử hệ thống có 4 node ($n=4$), định danh là 0, 1, 2, 3. Trong đó, Node 0 là Master, các node còn lại là Slave. Giả sử giới hạn lỗi là $f=1$ và Node 0 là node Byzantine (kẻ tấn công).

Theo định lý Dolev-Strong, số vòng trao đổi cần thiết để đạt đồng thuận là $f+1=1+1=2$ vòng.

Vòng 1 (Round 1): Master (Node 0) bắt đầu gửi giá trị. Vì là nút Byzantine, nó thực hiện tấn công “Gửi mâu thuẫn” (Equivocation):

- Gửi thông điệp đã ký $\langle 1:0 \rangle$ cho Node 1.
- Gửi thông điệp đã ký $\langle 0:0 \rangle$ cho Node 2.
- Không gửi gì cho Node 3 (lỗi bỏ sót).

Kết thúc Vòng 1: Tập giá trị của các node là: $V_1 = \{1\}$, $V_2 = \{0\}$, $V_3 = \emptyset$. Lúc này tính thỏa thuận (Agreement) chưa đạt được.

Vòng 2 (Round 2): Các node slave thực hiện quy tắc lan truyền nghiêm ngặt:

- Node 1: Nhận thấy $\langle 1:0 \rangle$ hợp lệ ở vòng 1, nó ký tiếp và phát tán $\langle 1:0,1 \rangle$ cho Node 2 và Node 3.
- Node 2: Nhận thấy $\langle 0:0 \rangle$ hợp lệ ở vòng 1, nó ký tiếp và phát tán $\langle 0:0,2 \rangle$ cho Node 1 và Node 3.

3.

• Node 3: Lắng nghe và nhận được cả hai thông điệp $\langle 1 : 0, 1 \rangle$ và $\langle 0 : 0, 2 \rangle$.

Kết thúc Vòng 2:

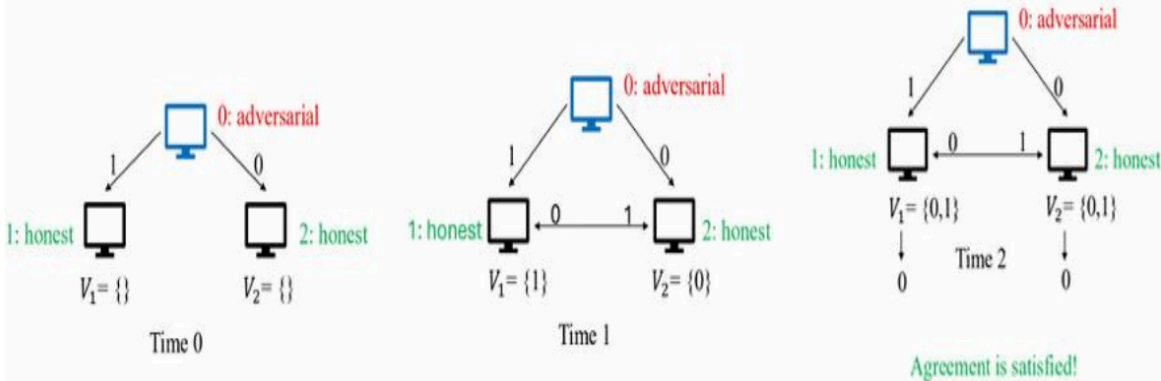
• Node 1 nhận thêm giá trị 0 từ Node 2 $\rightarrow V_1 = \{1, 0\}$.

• Node 2 nhận thêm giá trị 1 từ Node 1 $\rightarrow V_2 = \{0, 1\}$.

• Node 3 nhận được cả hai $\rightarrow V_3 = \{1, 0\}$.

Kết quả: Tất cả các node tin cậy (1, 2, 3) đều có cùng tập giá trị $\{0, 1\}$. Bằng cách áp dụng một quy tắc quyết định chung (ví dụ: chọn giá trị nhỏ nhất), tất cả sẽ cùng đồng thuận một kết quả, thỏa mãn tính Agreement.

• **Vẽ sơ đồ trao đổi thông điệp của giao thức Strawman 2 với 3 node slave và giải thích tại sao vòng 2 giúp đạt được Agreement . Trả lời:**



Sơ đồ trao đổi thông điệp (3 bước):

Bước 1 (Time0): Node Master (Node 0) gửi thông điệp có chữ ký cho các Slave:

$0 \rightarrow 1: \langle v : 0 \rangle$

$0 \rightarrow 2: \langle v : 0 \rangle$

$0 \rightarrow 3: \langle v : 0 \rangle$

Bước 2 (Time1): Các Slave nhận tin, cập nhật tập V_i , ký tên mình và phát lại (relay) cho các Slave khác:

Node 1 gửi cho (2, 3): $\langle v : 0, 1 \rangle$

Node 2 gửi cho (1, 3): $\langle v : 0, 2 \rangle$

Node 3 gửi cho (1, 2): $\langle v : 0, 3 \rangle$

Bước 3 (Time2): Các node thu thập tất cả thông điệp nhận được ở vòng 2, cập nhật tập V_i và đưa ra quyết định cuối cùng.

Tại sao Vòng 2 giúp đạt được Agreement? Trong Strawman 1, nếu Master gửi giá trị 1 cho Node 1 và giá trị 0 cho Node 2, hai node này sẽ quyết định khác nhau ngay lập tức, dẫn đến vi phạm tính thỏa thuận (Agreement).

Vòng 2 của Strawman 2 đóng vai trò là cơ chế “lan truyền xác thực”:

Chia sẻ thông tin: Ngay cả khi Master tìm cách lừa dối bằng cách gửi các giá trị mâu thuẫn cho các nút slave tin cậy, các nút slave tin cậy sẽ “tố giác” hành vi này bằng cách gửi giá trị mình nhận được cho toàn bộ mạng lưới.

Đồng bộ hóa tập dữ liệu: Nhờ việc trao đổi chéo ở Vòng 2, tất cả các nút slave tin cậy cuối cùng sẽ sở hữu cùng một tập hợp giá trị V_i mà Master đã phát ra.

Nhất quán quyết định: Khi mọi nút đều nhìn thấy cùng một tập dữ liệu, chúng sẽ áp dụng cùng một logic để đưa ra quyết định giống nhau, từ đó đạt được tính Agreement. Đây là giải pháp cho vấn đề Master gửi không đều hoặc gửi mâu thuẫn.

CÂU HỎI LÝ THUYẾT - TRONG FILE ÔN TẬP:

Trình bày ba tính chất Agreement, Validity, Termination của giao thức đồng thuận.

đó nếu có 3 node gian lận ($f = 3$) thì không còn đảm bảo an toàn, vì nhóm gian lận có thể làm các node trung thực nhận các thông điệp/phiếu bầu mâu thuẫn, dẫn đến việc các node trung thực có thể “commit” các giá trị khác nhau hoặc không thể thống nhất đúng cách.

DẠNG BÀI TẬP: Smart Contract

- Cấu trúc smart contract (trong file B4):
- Định nghĩa Giao diện (Interface) hoặc Kế thừa (Inheritance)
 - Biến Trạng thái (State Variables): lưu trữ dữ liệu tồn tại bền vững trên blockchain. Ví dụ: thông tin quản lý, thông tin cấu hình, dữ liệu nghiệp vụ.
 - Hàm Khởi tạo (Constructor): thiết lập trạng thái ban đầu cho hợp đồng.
 - Các Hàm (Functions):
 - Hàm ghi dữ liệu (thay đổi trạng thái), hàm đọc dữ liệu (không thay đổi trạng thái)
 - Tầm vực: external, public, internal, private
 - Sự kiện: ghi log lên blockchain, cho phép các ứng dụng bên ngoài theo dõi và phản hồi lại các hành động xảy ra trong hợp đồng

I. GIAO DIỆN (INTERFACE) & KẾ THỪA (INHERITANCE)

Nội dung	Câu hỏi thường gặp	Kiến thức cần nhớ	Cách trả lời chuẩn
Interface	Interface dùng để làm gì?	Định nghĩa hàm, không cài đặt	Interface mô tả các hàm mà contract phải tuân theo
Interface trong Vyper	Vyper có interface không?	Có, dùng interface	Dùng để tương tác contract khác
Kế thừa	Kế thừa dùng để làm gì?	Tái sử dụng code	Giảm trùng lặp, tăng bảo trì
ERC Interface	Vì sao cần IERC20 / IERC721?	Chuẩn hóa	Đảm bảo tương thích hệ sinh thái
implements	Từ khóa implements nghĩa là gì?	Cam kết triển khai	Contract phải cài đủ hàm interface
Lợi ích	Vì sao dùng interface?	Chuẩn – mở rộng	Dễ tích hợp, kiểm tra

II. BIẾN TRẠNG THÁI (STATE VARIABLES)

Nội dung	Câu hỏi thường gặp	Kiến thức cần nhớ	Cách trả lời chuẩn
State variable	Biến nào lưu trên blockchain?	Biến trạng thái	Lưu bền vững, tốn gas
Thông tin quản lý	Thường lưu gì?	admin / owner	Quản lý quyền
Thông tin cấu hình	Lưu gì?	tham số hệ thống	Cấu hình hợp đồng
Dữ liệu nghiệp vụ	Ví dụ?	balances, docs	Phục vụ bài toán
HashMap	Vì sao dùng?	Key–value	Truy cập nhanh
struct	Dùng khi nào?	Gom nhiều trường	Lưu record
exists: bool	Tại sao cần?	Phân biệt tồn tại	Tránh ghi đè
Gas	Biến nào tốn gas?	Struct + mapping	Ghi nhiều slot

III. HÀM KHỞI TẠO (CONSTRUCTOR)

Nội dung	Câu hỏi thường gặp	Kiến thức cần nhớ	Cách trả lời chuẩn
Constructor	Constructor là gì?	Hàm khởi tạo	Chạy 1 lần khi deploy
<code>__init__</code>	Khi nào chạy?	Lúc deploy	Không gọi lại được
Mục đích	Dùng để làm gì?	Thiết lập state	Gán admin, giá trị ban đầu
<code>msg.sender</code>	Vai trò?	Người deploy	Thường là owner
Thiếu constructor	Hậu quả?	State rỗng	Dễ lỗi logic
Bảo mật	Vì sao quan trọng?	Thiết lập quyền	Gốc của access control

IV. CÁC HÀM (FUNCTIONS) IV.1. Phân loại theo chức năng

Loại hàm	Câu hỏi thường gặp	Kiến thức cần nhớ	Cách trả lời chuẩn
Hàm ghi	Hàm nào ghi dữ liệu?	Thay đổi state	Tốn gas
Hàm đọc	Hàm nào chỉ đọc?	Không đổi state	Không tốn gas off-chain
Ví dụ ghi	transfer, mint	Cập nhật mapping	Logic nghiệp vụ
Ví dụ đọc	getBalance	<code>@view</code>	Truy vấn dữ liệu

IV.2. Phân loại theo tầm vực (visibility)

Tầm vực	Câu hỏi thường gặp	Kiến thức cần nhớ	Cách trả lời chuẩn
<code>external</code>	Khi nào dùng?	Gọi từ ngoài	Phổ biến nhất
<code>public</code>	Khác external?	Gọi cả trong & ngoài	Linh hoạt
<code>internal</code>	Ai gọi được?	Trong contract	Tái sử dụng logic
<code>private</code>	Ai gọi được?	Chỉ trong contract	Ẩn logic
So sánh	Vì sao chọn đúng tầm vực?	Bảo mật	Tránh gọi sai

V. SỰ KIỆN (EVENTS)

Nội dung	Câu hỏi thường gặp	Kiến thức cần nhớ	Cách trả lời chuẩn
Event	Event là gì?	Ghi log	Không lưu state
Mục đích	Dùng khi nào?	Theo dõi hành động	Frontend lắng nghe
Event vs State	Khác nhau?	Log vs lưu	Event rẻ gas
Indexed	Từ khóa indexed?	Tìm kiếm nhanh	Lọc log
Ví dụ	Event Transfer	Theo dõi giao dịch	Chuẩn ERC
Bỏ event	Hậu quả?	Mất tracking	UX kém

VII. CÂU TỔNG HỢP HAY RA (THEO B4)

- Trình bày cấu trúc một smart contract Ý chấm điểm: Interface → State → Constructor → Functions → Events
- Vì sao phải tách hàm đọc và ghi? Ý chấm điểm: Gas + an toàn

[illegible]

```

1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
181
```

```
self.balanceOf[src] -= wad
```

```
self.balanceOf[dst] += wad
```

Số dư hiện tại của hợp đồng WETH đơn giản này là hơn ba triệu ETH. Tức là, hơn ba triệu ETH đã được đóng gói dưới dạng WETH.

1. Giải thích cách Bob có thể lạm dụng hàm withdraw để rút được nhiều tiền hơn số tài khoản anh ta đang có. Giải thích cách Bob khai thác. Bob có thể khai thác hàm withdraw do hợp đồng vi phạm nguyên tắc Checks–Effects–Interactions, dẫn đến lỗ hổng reentrancy. Cụ thể, khi Bob gọi withdraw, hợp đồng gửi ETH ra ngoài cho Bob. Nếu Bob là một hợp đồng độc hại có hàm fallback, thì trong quá trình nhận ETH, hàm fallback có thể gọi lại withdraw nhiều lần liên tiếp trước khi quá trình rút tiền hoàn tất. Do trạng thái hợp đồng chưa được bảo vệ đầy đủ, Bob có thể tiếp tục rút ETH nhiều lần chỉ với một số dư WETH ban đầu, từ đó rút được nhiều ETH hơn số tiền anh ta thực sự sở hữu.

2. Các em đề xuất cách sửa hàm withdraw. Nêu ra lỗi và nguyên nhân lỗi sau đó thực hiện sửa lỗi. Lỗi trong hàm withdraw xuất phát từ việc tương tác với bên ngoài khi trạng thái nội bộ chưa được khóa an toàn. Để sửa lỗi, hợp đồng cần tuân thủ nguyên tắc Checks–Effects–Interactions, tức là phải kiểm tra điều kiện và cập nhật số dư WETH của người dùng trước, sau đó mới thực hiện gửi ETH ra ngoài. Ngoài ra, có thể bổ sung cơ chế reentrancy guard để ngăn chặn việc gọi lặp hàm withdraw. Nhờ đó, hợp đồng sẽ không cho phép người dùng rút ETH nhiều lần từ cùng một số dư WETH.

3. Có một lỗi nghiêm trọng trong hàm transferFrom. Giả sử Alice ủy quyền cho Bob chuyển 1 ETH từ tài khoản của cô ấy. Hãy giải thích cách Bob có thể chuyển nhiều hơn 1 ETH từ tài khoản của Alice. Bob có thể khai thác hàm transferFrom do xử lý không chặt chẽ cơ chế ủy quyền (allowance). Mặc dù Alice chỉ cho phép Bob chuyển 1 ETH, nhưng hợp đồng không đảm bảo rằng allowance được cập nhật và giới hạn một cách an toàn sau mỗi lần chuyển. Bob có thể gọi transferFrom nhiều lần liên tiếp và mỗi lần đều vượt qua điều kiện kiểm tra, từ đó chuyển nhiều ETH hơn số lượng Alice đã cho phép ban đầu. Đây là một lỗi logic nghiêm trọng trong việc triển khai chuẩn ERC-20.

4. Các em đề xuất sửa hàm transferFrom. Để sửa lỗi, hợp đồng cần đảm bảo rằng allowance được kiểm tra và giảm chính xác sau mỗi lần chuyển. Việc trừ allowance và cập nhật số dư tài khoản phải được thực hiện đầy đủ trước khi kết thúc hàm. Ngoài ra, thứ tự xử lý cần tuân thủ nguyên tắc Checks–Effects–Interactions để tránh các hành vi khai thác logic. Khi đó, Bob sẽ không thể sử dụng quyền ủy quyền để chuyển nhiều ETH hơn mức Alice cho phép.

5. Giả sử cả hai lỗi này đều đã được khắc phục. Vậy có luôn đúng rằng số dư ETH của hợp đồng WETH bằng tổng tất cả các nguồn vào trong balanceOf không? Tức là, hàm totalSupply có luôn trả về tổng số của tài khoản chính xác không? Không. Ngay cả khi cả hai lỗi trên đã được khắc phục, số dư ETH thực tế của hợp đồng WETH không phải lúc nào cũng bằng tổng các giá trị trong balanceOf. Nguyên nhân là do ETH có thể được gửi vào hợp đồng thông minh thông qua các cơ chế ngoài dự kiến, chẳng hạn như selfdestruct từ một hợp đồng khác. Những ETH này làm tăng số dư thực tế của hợp đồng nhưng không được ghi nhận trong balanceOf. Vì vậy, hàm totalSupply có thể không phản ánh chính xác tổng số ETH thực sự mà hợp đồng đang nắm giữ.

Dạng 1: Cho một đoạn chương trình smart contract viết bằng Vyper. Trả lời câu hỏi?

Bài 1 - trong slide B3: Cho một đoạn chương trình tạo smart contract bằng ngôn ngữ lập trình Vyper:

```
# @version ^0.4.3
```

```
owner: public(address) # Địa chỉ nhà sản xuất phim
balances: public(HashMap[address, uint256]) # Sổ dư token của nhà đầu tư
totalSupply: public(uint256) # Tổng cung token
lastWithdrawDate: public(uint256) # Thời gian rút tiền lần cuối
```

```
@payable
```

```
@deploy
```

```
def __init__():
```

```
    self.owner = msg.sender
    # Lưu số vốn khởi tạo (nếu có)
    self.balances[self.owner] = msg.value
    self.totalSupply = self.balances[self.owner]
```

```
@external
```

```
def withdraw(amount: uint256):
```

```
    # Chỉ nhà sản xuất (owner) mới được rút tiền
    assert msg.sender == self.owner, "Only owner may withdraw"
    # Kiểm tra số tiền rút
    assert amount > 0 and amount <= self.balance, "Invalid amount"
    # Chỉ được rút 1 lần mỗi tháng
    assert block.timestamp > self.lastWithdrawDate + 30 * 86400, "Only once per month"
    self.lastWithdrawDate = block.timestamp
    # Lịch rút tiền: mỗi tháng chỉ được rút nhiều hơn 1 ether so với số lần trước
    maxAmount: uint256 = self.totalSupply = self.balance + as_wei_value(1, "ether")
    assert amount <= maxAmount, "Exceeds allowed amount"
    # Gửi tiền cho nhà sản xuất
    send(self.owner, amount)
```

```
@payable
```

```
@external
```

```
def __default__():
```

```
    # fallback: ghi nhận đóng góp
    self.balances[msg.sender] += msg.value
    self.totalSupply += msg.value
```



Câu hỏi

A) Số tiền tối đa có thể rút trong mỗi tháng đầu tiên là bao nhiêu, giả sử mỗi tháng rút tối đa?

• Tháng 1: ? • Tháng 2: ? • Tháng 3: ? • Tháng 4: ? Gọi: S = totalSupply (tổng vốn ghi sổ), B = self.balance (số ETH thật đang nằm trong contract). Khi không có “ether bất ngờ”, ta có: Sau khi rút tổng cộng W ETH thì B = S - W. Khi đó maxAmount = S - B + 1 = W + 1 (ether). Vì mỗi tháng rút tối đa, ta được chuỗi:

Tháng 1: W=0 $\Rightarrow$ max = 1 ETH

Tháng 2: sau tháng 1 rút 1 $\Rightarrow$ W=1 $\Rightarrow$ max = 2 ETH

Tháng 3: sau tháng 2 rút 2 $\Rightarrow$ W=3 $\Rightarrow$ max = 4 ETH

Tháng 4: sau tháng 3 rút 4 $\Rightarrow$ W=7 $\Rightarrow$ max = 8 ETH

B) Hãy mô tả một cách tấn công cho phép nhà sản xuất phim rút toàn bộ số tiền của dự án ngay trong tháng đầu tiên. Gọi ý: Nếu ai đó gửi ether đến địa chỉ contract trước khi contract được tạo, thì khi contract được deploy tại địa chỉ đó, biến self.balance sẽ bằng với số ether đã gửi trước đó. Lưu ý: trong Vyper, các biến là uint256, nên giá trị -1 thực chất là $(2^{256}) - 1$. Ý

tưởng: gửi ETH vào “địa chỉ tương lai” của contract trước khi deploy (ví dụ dùng CREATE2 để biết trước địa chỉ, hoặc bằng cơ chế mà đề bài gợi ý). Khi deploy xong:

self.balance = msg.value + X (X là ETH đã bị gửi vào địa chỉ đó từ trước)

Nhưng trong __init__ hiện tại: self.totalSupply = self.balances[self.owner] = msg.value $\Rightarrow$ totalSupply = msg.value không tính X

Khi đó:

self.totalSupply - self.balance là số âm, nhưng kiểu uint256 sẽ underflow thành một số cực lớn $(2^{256} - (self.balance - self.totalSupply))$.

Nên maxAmount trở thành cực lớn, điều kiện amount <= maxAmount luôn đúng.

Lại có check amount <= self.balance (ETH thật trong contract), nên owner chỉ cần gọi:

withdraw(self.balance) ngay tháng đầu tiên là rút sạch.

Bài 2: trong slide B4:

```
userBalances: public(HashMap[address, uint256])
```

```
@external
@view
def getUserBalance(user: address) -> uint256:
    return self.userBalances[user]

@external
@payable
def addToBalance():
    self.userBalances[msg.sender] += msg.value

@external
def withdrawBalance():
    amountToWithdraw: uint256 = self.userBalances[msg.sender]

    success: bool = raw_call(
        msg.sender,
        b"",
        value=amountToWithdraw,
        gas=msg.gas
    )

    if not success:
        raise "Transfer failed"

    self.userBalances[msg.sender] = 0
```

```
bank: public(Bank)
numIterations: public(uint256)

@external
@payable
def __init__(_bankAddress: address):
    self.bank = Bank(_bankAddress)
    self.numIterations = 10

    # Deposit 75 Wei
    self.bank.addToBalance(value=75)

    # Trigger attack
    self.bank.withdrawBalance()

@external
@payable
def __default__():
    # Vyper's fallback function
    if self.numIterations > 0:
        self.numIterations -= 1
        self.bank.withdrawBalance()
```

Phân tích lỗ hổng Reentrancy -TRONG FILE B4 - 46,47 • Tình huống: Hợp đồng Bank trong slide có lỗ hổng reentrancy.

• Yêu cầu: ▪ Vẽ sơ đồ luồng tấn công của hacker.

```
Attacker.deposit(X)
Attacker.withdraw()
Bank: amount = X
Bank -> Attacker (send X) ---- external call ----
Attacker.fallback():
    Attacker.withdraw() (re-enter)
    Bank: amount = X (chưa reset)
    Bank -> Attacker (send X)
    Attacker.fallback() ...
```

▪ **Giải thích tại sao cập nhật số dư sau khi gửi tiền lại gây ra lỗ hổng.** Vì bạn đã vi phạm Checks–Effects–Interactions: Checks: kiểm tra điều kiện (ví dụ amount > 0). Effects: cập nhật trạng thái nội bộ (đặt balance = 0 / trừ tiền). Interactions: gọi ra ngoài (gửi ETH / gọi contract khác) Trong code slide: Bank tương tác ra ngoài trước (raw_call(...)). Trong lúc đó, attacker có quyền chạy code (fallback) và gọi lại Bank. Vì trạng thái userBalances[msg.sender] chưa bị giảm/clear, attacker đọc lại vẫn thấy X và rút tiếp X. Điểm “chết người” là gas=msg.gas + raw_call ⇒ forward rất nhiều gas, fallback chạy thoải mái để gọi lại.

▪ Viết lại hợp đồng (bằng Solidity hoặc Vyper) để khắc phục lỗi bằng Checks-Effects-Interaction:

```
# @version ^0.4.0
userBalances: public(HashMap[address, uint256])

@external
@view
def getUserBalance(user: address) -> uint256:
    return self.userBalances[user]
```

```

@external
@payable
def addToBalance():
    self.userBalances[msg.sender] += msg.value
@external
def withdrawBalance():
    amount: uint256 = self.userBalances[msg.sender]
    # Checks
    assert amount > 0, "No balance"
    # Effects (UPDATE TRƯỚC)
    self.userBalances[msg.sender] = 0
    # Interactions (GỬI SAU)
    success: bool = raw_call(
        msg.sender,
        b"",
        value=amount,
        gas=msg.gas,
    )
    if not success:
        # nếu chuyển thất bại: rollback trạng thái
        self.userBalances[msg.sender] = amount
        raise "Transfer failed"

```

Triển khai Reentrancy Guard • Tình huống: Bạn cần nâng cấp hợp đồng rút tiền an toàn hơn. •

Yêu cầu:

▪ Sử dụng kỹ thuật Reentrancy Guard (mutex) để viết lại hàm withdrawBalance().

```

# @version ^0.4.0
# Bank contract with Reentrancy Guard (mutex)
userBalances: public(HashMap[address, uint256])
locked: bool # mutex reentrancy guard
@external
@view
def getUserBalance(user: address) -> uint256:
    return self.userBalances[user]
@external
@payable
def addToBalance():
    # Optional: prevent useless deposits
    assert msg.value > 0, "Zero deposit"
    self.userBalances[msg.sender] += msg.value
@external
def withdrawBalance():
    # --- Reentrancy Guard (mutex) ---
    assert not self.locked, "Reentrancy blocked"
    self.locked = True
    amount: uint256 = self.userBalances[msg.sender]
    assert amount > 0, "No balance"
    # Effects: update state BEFORE external call (still good practice)
    self.userBalances[msg.sender] = 0
    # Interaction: send ETH out (may trigger fallback on receiver)
    success: bool = raw_call(

```



```
def deposit():
```

```
    assert self.balances[msg.sender] <= max_value(uint256) - msg.value
```

```
    self.balances[msg.sender] += msg.value
```

Cách sửa B (khuyến nghị nếu dùng Vyper mới): Với các bản Vyper mới, overflow/underflow thường đã bị revert mặc định (trừ khi bạn dùng các hàm `unsafe_*`). Nhưng khi làm bài/thi, cứ viết `assert` chống tràn như trên là rõ ràng nhất.

Bonus (2 lỗi “ngân hàng” hay bị hỏi kèm, dù đề đang nhầm Overflow/Underflow): Không có event `Deposit/Withdraw` → khó audit.

Không có `assert _amount > 0` (không nghiêm trọng nhưng nên có). `send()` kiểu “transfer-like” thường hạn chế reentrancy, nhưng nếu đòi sang call forward gas thì phải thêm reentrancy guard.

Dạng 2: Miêu tả bài toán, yêu cầu viết smart contract:

Thiết kế hợp đồng thông minh đơn giản

- **Tình huống:** Công ty ABC muốn tạo một Hợp đồng Lưu trữ Thông điệp Chúc mừng trên blockchain Ethereum. Hợp đồng này cho phép:

- Quản trị viên (admin) thiết lập thông điệp chúc mừng mặc định khi triển khai hợp đồng
- Bất kỳ ai cũng có thể xem thông điệp hiện tại
- Chỉ quản trị viên mới có quyền cập nhật thông điệp mới.

- **Yêu cầu:**

- Viết mã hợp đồng (Solidity hoặc Vyper) có các hàm: `setMessage(string newMessage)` và `getMessage()`.
- Chỉ rõ các thành phần: biến trạng thái, constructor (nếu cần), hàm `public/external`.

```
# @version ^0.4.0
```

```
# --- State Variables ---
```

```
admin: public(address)
```

```
message: String[256]
```

```
# --- Constructor ---
```

```
@deploy
```

```
def __init__(_initialMessage: String[256]):
```

```
    self.admin = msg.sender
```

```
    self.message = _initialMessage
```

```
# --- Get current message ---
```

```
@external
```

```
@view
```

```
def getMessage() -> String[256]:
```

```
    return self.message
```

```
# --- Update message (admin only) ---
```

```
@external
```

```
def setMessage(newMessage: String[256]):
```

```
    assert msg.sender == self.admin, "Only admin can update message"
```

```
    self.message = newMessage
```

a) **Biến trạng thái:** `admin: address` – lưu địa chỉ quản trị viên. `message: String[...]` – lưu thông điệp chúc mừng hiện tại.

b) **Constructor:** Hàm `__init__()`: Gán `admin = msg.sender`. Khởi tạo thông điệp ban đầu.

c) Hàm `getMessage()` – `external/view`, ai cũng gọi được. `setMessage(newMessage)` – `external`, chỉ admin được gọi.

DẠNG BÀI TẬP: Ứng dụng phi tập trung DApp:

- Phân tích, thiết kế ▪ Đọc code, trả lời câu hỏi

Bài 1 trên slide B4 - SLIDE 48:

Thiết kế hệ thống DApp đơn giản

- **Tình huống:** Công ty của bạn muốn xây dựng một DApp cho phép người dùng bỏ phiếu (voting) cho một đề xuất.
- **Yêu cầu:**
 - Thiết kế kiến trúc tổng quan DApp (smart contract + front-end).
 - Viết mã hợp đồng thông minh (Solidity/Vyper) với các chức năng: đăng ký đề xuất, bỏ phiếu, kiểm tra kết quả.
 - Đề xuất cơ chế bảo mật để tránh gian lận (ví dụ: mỗi địa chỉ chỉ được bỏ phiếu một lần).

1) Phân tích & thiết kế kiến trúc tổng quan DApp (Smart contract + Front-end): Kiến trúc 3 lớp (đúng mô hình DApp trong file Ethereum/DApp): Lớp Smart Contract (on-chain): lưu *proposal*, số phiếu, trạng thái đã vote, và thực thi logic bỏ phiếu minh bạch, chống sửa lịch sử. Front-end (off-chain UI): web app (React/HTML/JS) kết nối ví (MetaMask) để: tạo đề xuất, bỏ phiếu, xem kết quả (read-only). Kết nối Web3: Front-end gọi ABI của contract qua provider ([ethers.js/web3.js](https://ethers.js.org/web3.js)). Triển khai/kiểm thử: dùng testnet/local chain (Ganache/Geth/Hardhat) rồi deploy lên mạng thật.

2) Viết mã hợp đồng thông minh (Vyper – có đủ đăng ký đề xuất, bỏ phiếu, kiểm tra kết quả)
(Viết theo kiểu đơn giản, đúng yêu cầu “mỗi địa chỉ chỉ được bỏ phiếu một lần”).

```
# @version ^0.4.3
```

```
# ===== DATA STRUCTURES =====
```

```
struct Proposal:
```

```
    description: String[256]
```

```
    yesVotes: uint256
```

```
    noVotes: uint256
```

```
    exists: bool
```

```
# ===== STATE =====
```

```
owner: public(address)
```

```
proposalCount: public(uint256)
```

```
proposals: public(HashMap[uint256, Proposal])
```

```
hasVoted: public(HashMap[uint256, HashMap[address, bool]]) # proposalId -> voter -> voted?
```

```
# ===== EVENTS =====
```

```
event ProposalCreated:
```

```
    proposalId: uint256
```

```
    description: String[256]
```

```
event Voted:
```

```
    proposalId: uint256
```

```
    voter: address
```

```
    support: bool
```

```
# ===== CONSTRUCTOR =====
```

```
@deploy
```

```
def __init__():
```

```
    self.owner = msg.sender
```

```
    self.proposalCount = 0
```

```
# ===== 1) ĐĂNG KÝ ĐỀ XUẤT =====
```

```
@external
```

```
def createProposal(desc: String[256]) -> uint256:
```

```
    # (tùy chọn) chỉ owner tạo đề xuất:
```

```
    assert msg.sender == self.owner, "Only owner can create proposal"
```

```
    pid: uint256 = self.proposalCount
```

```
    self.proposals[pid] = Proposal({
```

```
        description: desc,
```

```
        yesVotes: 0,
```

```
        noVotes: 0,
```


1. Theo 1 Thimotee 2:16-17, 23-24, 28-29, 34-35, 38-39, 41-42, 44-45, 47-48, 50-51, 53-54, 56-57, 60-61, 63-64, 66-67, 69-70, 72-73, 75-76, 78-79, 81-82, 84-85, 87-88, 90-91, 93-94, 96-97, 99-100, 102-103, 105-106, 108-109, 111-112, 114-115, 117-118, 120-121, 123-124, 126-127, 129-130, 132-133, 135-136, 138-139, 141-142, 144-145, 147-148, 150-151, 153-154, 156-157, 159-160, 162-163, 165-166, 168-169, 171-172, 174-175, 177-178, 180-181, 183-184, 186-187, 189-190, 192-193, 195-196, 198-199, 201-202, 204-205, 207-208, 210-211, 213-214, 216-217, 219-220, 222-223, 225-226, 228-229, 231-232, 234-235, 237-238, 240-241, 243-244, 246-247, 249-250, 252-253, 255-256, 258-259, 261-262, 264-265, 267-268, 270-271, 273-274, 276-277, 279-280, 282-283, 285-286, 288-289, 291-292, 294-295, 297-298, 300-301, 303-304, 306-307, 309-310, 312-313, 315-316, 318-319, 321-322, 324-325, 327-328, 330-331, 333-334, 336-337, 339-340, 342-343, 345-346, 348-349, 351-352, 354-355, 357-358, 360-361, 363-364, 366-367, 369-370, 372-373, 375-376, 378-379, 381-382, 384-385, 387-388, 390-391, 393-394, 396-397, 399-400, 402-403, 405-406, 408-409, 411-412, 414-415, 417-418, 420-421, 423-424, 426-427, 429-430, 432-433, 435-436, 438-439, 441-442, 444-445, 447-448, 450-451, 453-454, 456-457, 459-460, 462-463, 465-466, 468-469, 471-472, 474-475, 477-478, 480-481, 483-484, 486-487, 489-490, 492-493, 495-496, 498-499, 501-502, 504-505, 507-508, 510-511, 513-514, 516-517, 519-520, 522-523, 525-526, 528-529, 531-532, 534-535, 537-538, 540-541, 543-544, 546-547, 549-550, 552-553, 555-556, 558-559, 561-562, 564-565, 567-568, 570-571, 573-574, 576-577, 579-580, 582-583, 585-586, 588-589, 591-592, 594-595, 597-598, 600-601, 603-604, 606-607, 609-610, 612-613, 615-616, 618-619, 621-622, 624-625, 627-628, 630-631, 633-634, 636-637, 639-640, 642-643, 645-646, 648-649, 651-652, 654-655, 657-658, 660-661, 663-664, 666-667, 669-670, 672-673, 675-676, 678-679, 681-682, 684-685, 687-688, 690-691, 693-694, 696-697, 699-700, 702-703, 705-706, 708-709, 711-712, 714-715, 717-718, 720-721, 723-724, 726-727, 729-730, 732-733, 735-736, 738-739, 741-742, 744-745, 747-748, 750-751, 753-754, 756-757, 759-760, 762-763, 765-766, 768-769, 771-772, 774-775, 777-778, 780-781, 783-784, 786-787, 789-790, 792-793, 795-796, 798-799, 801-802, 804-805, 807-808, 810-811, 813-814, 816-817, 819-820, 822-823, 825-826, 828-829, 831-832, 834-835, 837-838, 840-841, 843-844, 846-847, 849-850, 852-853, 855-856, 858-859, 861-862, 864-865, 867-868, 870-871, 873-874, 876-877, 879-880, 882-883, 885-886, 888-889, 891-892, 894-895, 897-898, 900-901, 903-904, 906-907, 909-910, 912-913, 915-916, 918-919, 921-922, 924-925, 927-928, 929-930, 932-933, 935-936, 938-939, 941-942, 944-945, 947-948, 950-951, 953-954, 956-957, 959-960, 962-963, 965-966, 968-969, 971-972, 974-975, 977-978, 980-981, 983-984, 986-987, 989-990, 992-993, 995-996, 998-999, 1000-1001, 1003-1004, 1006-1007, 1009-1010, 1012-1013, 1015-1016, 1018-1019, 1021-1022, 1024-1025, 1027-1028, 1030-1031, 1033-1034, 1036-1037, 1039-1040, 1042-1043, 1045-1046, 1048-1049, 1051-1052, 1054-1055, 1057-1058, 1060-1061, 1063-1064, 1066-1067, 1069-1070, 1072-1073, 1075-1076, 1078-1079, 1081-1082, 1084-1085, 1087-1088, 1090-1091, 1093-1094, 1096-1097, 1099-1100, 1102-1103, 1105-1106, 1108-1109, 1111-1112, 1114-1115, 1117-1118, 1120-1121, 1123-1124, 1126-1127, 1129-1130, 1132-1133, 1135-1136, 1138-1139, 1141-1142, 1144-1145, 1147-1148, 1150-1151, 1153-1154, 1156-1157, 1159-1160, 1162-1163, 1165-1166, 1168-1169, 1171-1172, 1174-1175, 1177-1178, 1180-1181, 1183-1184, 1186-1187, 1189-1190, 1192-1193, 1195-1196, 1198-1199, 1200-1201, 1203-1204, 1206-1207, 1209-1210, 1212-1213, 1215-1216, 1218-1219, 1221-1222, 1224-1225, 1227-1228, 1230-1231, 1233-1234, 1236-1237, 1239-1240, 1242-1243, 1245-1246, 1248-1249, 1251-1252, 1254-1255, 1257-1258, 1260-1261, 1263-1264, 1266-1267, 1269-1270, 1272-1273, 1275-1276, 1278-1279, 1281-1282, 1284-1285, 1287-1288, 1290-1291, 1293-1294, 1296-1297, 1299-1300, 1302-1303, 1305-1306, 1308-1309, 1311-1312, 1314-1315, 1317-1318, 1320-1321, 1323-1324, 1326-1327, 1329-1330, 1332-1333, 1335-1336, 1338-1339, 1341-1342, 1344-1345, 1347-1348, 1350-1351, 1353-1354, 1356-1357, 1359-1360, 1362-1363, 1365-1366, 1368-1369, 1371-1372, 1374-1375, 1377-1378, 1380-1381, 1383-1384, 1386-1387, 1389-1390, 1392-1393, 1395-1396, 1398-1399, 1401-1402, 1404-1405, 1407-1408, 1410-1411, 1413-1414, 1416-1417, 1419-1420, 1422-1423, 1425-1426, 1428-1429, 1431-1432, 1434-1435, 1437-1438, 1440-1441, 1443-1444, 1446-1447, 1449-1450, 1452-1453, 1455-1456, 1458-1459, 1461-1462, 1464-1465, 1467-1468, 1470-1471, 1473-1474, 1476-1477, 1479-1480, 1482-1483, 1485-1486, 1488-1489, 1491-1492, 1494-1495, 1497-1498, 1499-1500, 1502-1503, 1505-1506, 1508-1509, 1511-1512, 1514-1515, 1517-1518, 1520-1521, 1523-1524, 1526-1527, 1529-1530, 1532-1533, 1535-1536, 1538-1539, 1541-1542, 1544-1545, 1547-1548, 1550-1551, 1553-1554, 1556-1557, 1559-1560, 1562-1563, 1565-1566, 1568-1569, 1571-1572, 1574-1575, 1577-1578, 1580-1581, 1583-1584, 1586-1587, 1589-1590, 1592-1593, 1595-1596, 1598-1599, 1601-1602, 1604-1605, 1607-1608, 1610-1611, 1613-1614, 1616-1617, 1619-1620, 1622-1623, 1625-1626, 1628-1629, 1631-1632, 1634-1635, 1637-1638, 1640-1641, 1643-1644, 1646-1647, 1649-1650, 1652-1653, 1655-1656, 1658-1659, 1661-1662, 1664-1665, 1667-1668, 1670-1671, 1673-1674, 1676-1677, 1679-1680, 1682-1683, 1685-1686, 1688-1689, 1691-1692, 1694-1695, 1697-1698, 1699-1700, 1702-1703, 1705-1706, 1708-1709, 1711-1712, 1714-1715, 1717-1718, 1720-1721, 1723-1724, 1726-1727, 1729-1730, 1732-1733, 1735-1736, 1738-1739, 1741-1742, 1744-1745, 1747-1748, 1750-1751, 1753-1754, 1756-1757, 1759-1760, 1762-1763, 1765-1766, 1768-1769, 1771-1772, 1774-1775, 1777-1778, 1780-1781, 1783-1784, 1786-1787, 1789-1790, 1792-1793, 1795-1796, 1798-1799, 1801-1802, 1804-1805, 1807-1808, 1810-1811, 1813-1814, 1816-1817, 1819-1820, 1822-1823, 1825-1826, 1828-1829, 1831-1832, 1834-1835, 1837-1838, 1840-1841, 1843-1844, 1846-1847, 1849-1850, 1852-1853, 1855-1856, 1858-1859, 1861-1862, 1864-1865, 1867-1868, 1870-1871, 1873-1874, 1876-1877, 1879-1880, 1882-1883, 1885-1886, 1888-1889, 1891-1892, 1894-1895, 1897-1898, 1899-1900, 1902-1903, 1905-1906, 1908-1909, 1911-1912, 1914-1915, 1917-1918, 1920-1921, 1923-1924, 1926-1927, 1929-1930, 1932-1933, 1935-1936, 1938-1939, 1941-1942, 1944-1945, 1947-1948, 1950-1951, 1953-1954, 1956-1957, 1959-1960, 1962-1963, 1965-1966, 1968-1969, 1971-1972, 1974-1975, 1977-1978, 1980-1981, 1983-1984, 1986-1987, 1989-1990, 1992-1993, 1995-1996, 1998-1999, 2000-2001, 2003-2004, 2006-2007, 2009-2010, 2012-2013, 2015-2016, 2018-2019, 2021-2022, 2024-2025, 2027-2028, 2030-2031, 2033-2034, 2036-2037, 2039-2040, 2042-2043, 2045-2046, 2048-2049, 2051-2052, 2054-2055, 2057-2058, 2060-2061, 2063-2064, 2066-2067, 2069-2070, 2072-2073, 2075-2076, 2078-2079, 2081-2082, 2084-2085, 2087-2088, 2090-2091, 2093-2094, 2096-2097, 2099-2100, 2102-2103, 2105-2106, 2108-2109, 2111-2112, 2114-2115, 2117-2118, 2120-2121, 2123-2124, 2126-2127, 2129-2130, 2132-2133, 2135-2136, 2138-2139, 2141-2142, 2144-2145, 2147-2148, 2150-2151, 2153-2154, 2156-2157, 2159-2160, 2162-2163, 2165-2166, 2168-2169, 2171-2172, 2174-2175, 2177-2178, 2180-2181, 2183-2184, 2186-2187, 2189-2190, 2192-2193, 2195-2196, 2198-2199, 2201-2202, 2204-2205, 2207-2208, 2210-2211, 2213-2214, 2216-2217, 2219-2220, 2222-2223, 2225-2226, 2228-2229, 2231-2232, 2234-2235, 2237-2238, 2240-2241, 2243-2244, 2246-2247, 2249-2250, 2252-2253, 2255-2256, 2258-2259, 2261-2262, 2264-2265, 2267-2268, 2270-2271, 2273-2274, 2276-2277, 2279-2280, 2282-2283, 2285-2286, 2288-2289, 2291-2292, 2294-2295, 2297-2298, 2299-2300, 2302-2303, 2305-2306, 2308-2309, 2311-2312, 2314-2315, 2317-2318, 2320-2321, 2323-2324, 2326-2327, 2329-2330, 2332-2333, 2335-2336, 2338-2339, 2341-2342, 2344-2345, 2347-2348, 2350-2351, 2353-2354, 2356-2357, 2359-2360, 2362-2363, 2365-2366, 2368-2369, 2371-2372, 2374-2375, 2377-2378, 2380-2381, 2383-2384, 2386-2387, 2389-2390, 2392-2393, 2395-2396, 2398-2399, 2401-2402, 2404-2405, 2407-2408, 2410-2411, 2413-2414, 2416-2417, 2419-2420, 2422-2423, 2425-2426, 2428-2429, 2431-2432, 2434-2435, 2437-2438, 2440-2441, 2443-2444, 2446-2447, 2449-2450, 2452-2453, 2455-2456, 2458-2459, 2461-2462, 2464-2465, 2467-2468, 2470-2471, 2473-2474, 2476-2477, 2479-2480, 2482-2483, 2485-2486, 2488-2489, 2491-2492, 2494-2495, 2497-2498, 2499-2500, 2502-2503, 2505-2506, 2508-2509, 2511-2512, 2514-2515, 2517-2518, 2520-2521, 2523-2524, 2526-2527, 2529-2530, 2532-2533, 2535-2536, 2538-2539, 2541-2542, 2544-2545, 2547-2548, 2550-2551, 2553-2554, 2556-2557, 2559-2560, 2562-2563, 2565-2566, 2568-2569, 2571-2572, 2574-2575, 2577-2578, 2580-2581, 2583-2584, 2586-2587, 2589-2590, 2592-2593, 2595-2596, 2598-2599, 2601-2602, 2604-2605, 2607-2608, 2610-2611, 2613-2614, 2616-2617, 2619-2620, 2622-2623, 2625-2626, 2628-2629, 2631-2632, 2634-2635, 2637-2638, 2640-2641, 2643-2644, 2646-2647, 2649-2650, 2652-2653, 2655-2656, 2658-2659, 2661-2662, 2664-2665, 2667-2668, 2670-2671, 2673-2674, 2676-2677, 2679-2680, 2682-2683, 2685-2686, 2688-2689, 2691-2692, 2694-2695, 2697-2698, 2699-2700, 2702-2703, 2705-2706, 2708-2709, 2711-2712, 2714-2715, 2717-2718, 2720-2721, 2723-2724, 2726-2727, 2729-2730, 2732-2733, 2735-2736, 2738-2739, 2741-2742, 2744-2745, 2747-2748, 2750-2751, 2753-2754, 2756-2757, 2759-2760, 2762-2763, 2765-2766, 2768-2769, 2771-2772, 2774-2775, 2777-2778, 2780-2781, 2783-2784, 2786-2787, 2789-2790, 2792-2793, 2795-2796, 2798-2799, 2801-2802, 2804-2805, 2807-2808, 2810-2811, 2813-2814, 2816-2817, 2819-2820, 2822-2823, 2825-2826, 2828-2829, 2831-2832, 2834-2835, 2837-2838, 2840-2841, 2843-2844, 2846-2847, 2849-2850, 2852-2853, 2855-2856, 2858-2859, 2861-2862, 2864-2865, 2867-2868, 2870-2871, 2873-2874, 2876-2877, 2879-2880, 2882-2883, 2885-2886, 2888-2889, 2891-2892, 2894-2895, 2897-2898, 2899-2900, 2902-2903, 2905-2906, 2908-2909, 2911-2912, 2914-2915, 2917-2918, 2920-2921, 2923-2924, 2926-2927, 2929-2930, 2932-2933, 2935-2936, 2938-2939, 2941-2942, 2944-2945, 2947-2948, 2950-2951, 2953-2954, 2956-2957, 2959-2960, 2962-2963, 2965-2966, 2968-2969, 2971-2972, 2974-2975, 2977-2978, 2980-2981, 2983-2984, 2986-2987, 2989-2990, 2992-2993, 2995-2996, 2998-2999, 3000-3001, 3003-3004, 3006-3007, 3009-3010, 3012-3013, 3015-3016, 3018-3019, 3021-3022, 3024-3025, 3027-3028, 3030-3031, 3033-3034, 3036-3037, 3039-3040, 3042-3043, 3045-3046, 3048-3049, 3051-3052, 3054-3055, 3057-3058, 3060-3061, 3063-3064, 3066-3067, 3069-3070, 3072-3073, 3075-3076, 3078-3079, 3081-3082, 3084-3085, 3087-3088, 3090-3091, 3093-3094, 3096-3097, 3099-3100, 3102-3103, 3105-3106, 3108-3109, 3111-3112, 3114-3115, 3117-3118, 3120-3121, 3123-3124, 3126-3127, 3129-3130, 3132-3133, 3135-3136, 3138-3139, 3141-3142, 3144-3145, 3147-3148, 3150-3151, 3153-3154, 3156-3157, 3159-3160, 3162-3163, 3165-3166, 3168-3169, 3171-3172, 3174-3175, 3177-3178, 3180-3181, 3183-3184, 3186-3187, 3189-3190, 3192-3193, 3195-3196, 3198-3199, 3201-3202, 3204-3205, 3207-3208, 3210-3211, 3213-3214, 3216-3217, 3219-3220, 3222-3223, 3225-3226, 3228-3229, 3231-3232, 3234-3235, 3237-3238, 3240-3241, 3243-3244, 3246-3247, 3249-3250, 3252-3253, 3255-3256, 3258-3259, 3261-3262, 3264-3265, 3267-3268, 3270-3271, 3273-3274, 3276-3277, 3279-3280, 3282-3283, 3285-3286, 3288-3289, 3291-3292, 3294-3295, 3297-3298, 3299-3300, 3302-3303, 3305-3306, 3308-3309, 3311-3312, 3314-3315, 3317-3318, 3320-3321, 3323-3324, 3326-3327, 3329-3330, 3332-3333, 3335-3336, 3338-3339, 3341-3342, 3344-3345, 3347-3348, 3350-3351, 3353-3354, 3356-3357, 3359-3360, 3362-3363, 3365-3366, 3368-3369, 3371-3372, 3374-3375, 3377-3378, 3380-3381, 3383-3384, 3386-3387, 3389-3390, 3392-3393, 3395-3396, 3398-3399, 3401-3402, 3404-3405, 3407-3408, 3410-3411, 3413-3414, 3416-3417, 3419-3420, 3422-3423, 3425-3426, 3428-3429, 3431-3432, 3434-3435, 3437-3438, 3440-3441, 3443-3444, 3446-3447, 3449-3450, 3452-3453, 3455-3456, 3458-3459, 3461-3462, 3464-3465, 3467-3468, 3470-3471, 3473-3474, 3476-3477, 3479-3480, 3482-3483, 3485-3486, 3488-3489, 3491-3492, 3494-3495, 3497-3498, 3499-3500, 3502-3503, 3505-3506, 3508-3509, 3511-3512, 3514-3515, 3517-3518, 3520-3521, 3523-3524, 3526-3527, 3529-3530, 3532-3533, 3535-3536, 3538-3539, 3541-3542, 3544-3545, 3547-3548, 3550-3551, 35

Điểm xuất phát của các nhà nghiên cứu về biến đổi khí hậu (1980 – 1990) được coi là một trong những bước khởi đầu quan trọng của các nhà khoa học trong việc nghiên cứu biến đổi khí hậu. Các nhà nghiên cứu đã bắt đầu tập trung vào việc nghiên cứu các yếu tố ảnh hưởng đến biến đổi khí hậu, bao gồm cả các yếu tố tự nhiên và nhân tạo. Các nghiên cứu này đã giúp chúng ta hiểu rõ hơn về biến đổi khí hậu và tầm quan trọng của việc nghiên cứu biến đổi khí hậu.

[illegible]

Đến nay, các chuyên gia vẫn chưa thể xác định được chính xác nguyên nhân đang thúc đẩy bệnh viêm khớp dạng thấp (RA) tiến triển nhanh hay chậm. Nhưng có một điều chắc chắn là: Nếu không được điều trị đúng cách, bệnh này có thể gây ra những biến chứng nghiêm trọng, thậm chí đe dọa tính mạng người bệnh. Vì vậy, người bệnh cần phải được chẩn đoán sớm và điều trị đúng cách để ngăn ngừa biến chứng xảy ra.

[illegible]

1. **What does the sentence "Theater is 'artificial' communication" mean?**
 It means that theater is a constructed or staged form of communication, not a natural or spontaneous one. It involves deliberate choices in language, action, and setting to create a specific experience for the audience.

[illegible]

ĐỀ THI THỬ THPT QUỐC GIA NĂM 2019 MÔN THI KHOA HỌC TỰ NHIÊN (10/05/2019) [\[TẢI ĐỀ THI\]](#)
 Đề thi được tổ chức theo hình thức thi trắc nghiệm với 40 câu hỏi và đáp án đúng/sai. Đề thi được
 chia thành 2 phần: Phần I: Kiến thức cơ bản (20 câu) và Phần II: Kiến thức nâng cao (20 câu).
 Thời gian làm bài: 90 phút. Đề thi được tổ chức thi thử tại các trường THPT trên toàn quốc.
 Đề thi được tổ chức thi thử tại các trường THPT trên toàn quốc.

[illegible]

© 2011 The Authors
Journal compilation © 2011 Blackwell Publishing Ltd, *Journal of Internal Medicine* 270: 103–110

© 2000 Blackwell Science Ltd, *Journal of Internal Medicine* 247: 395–402

Information on the use of the model is available in the literature. The model is used to estimate the probability of a person being a victim of a crime, given the characteristics of the person and the location of the crime. The model is used to estimate the probability of a person being a victim of a crime, given the characteristics of the person and the location of the crime.

1. The 1st stage of the process is the initial stage, which is the first stage of the process. It is the first stage of the process, which is the first stage of the process.

2. The 2nd stage of the process is the second stage, which is the second stage of the process. It is the second stage of the process, which is the second stage of the process.

3. The 3rd stage of the process is the third stage, which is the third stage of the process. It is the third stage of the process, which is the third stage of the process.

4. The 4th stage of the process is the fourth stage, which is the fourth stage of the process. It is the fourth stage of the process, which is the fourth stage of the process.

5. The 5th stage of the process is the fifth stage, which is the fifth stage of the process. It is the fifth stage of the process, which is the fifth stage of the process.

6. The 6th stage of the process is the sixth stage, which is the sixth stage of the process. It is the sixth stage of the process, which is the sixth stage of the process.

7. The 7th stage of the process is the seventh stage, which is the seventh stage of the process. It is the seventh stage of the process, which is the seventh stage of the process.

8. The 8th stage of the process is the eighth stage, which is the eighth stage of the process. It is the eighth stage of the process, which is the eighth stage of the process.

9. The 9th stage of the process is the ninth stage, which is the ninth stage of the process. It is the ninth stage of the process, which is the ninth stage of the process.

10. The 10th stage of the process is the tenth stage, which is the tenth stage of the process. It is the tenth stage of the process, which is the tenth stage of the process.

11. The 11th stage of the process is the eleventh stage, which is the eleventh stage of the process. It is the eleventh stage of the process, which is the eleventh stage of the process.

